Lecture 05.10: Practice Problem (In-Class Banking System)

📌 Executive Summary

End-to-end implementation of an encapsulated banking system: deposits, withdrawals with overdraft protection, audit logs, and transaction invariants.

🔬 Theoretical Foundations & Principles

1. Financial Domain Invariants

A financial ledger must enforce strict accounting invariants:

1. Balance cannot drop below the allowable credit limit (overdraft constraint).
2. Transactions must be atomic and recorded in an immutable audit sequence.
3. Monetary amounts must be strictly positive ($x > 0$).

💻 Production Implementation & Code

```
class BankAccount:
    def __init__(self, account_number: str, owner: str, initial_balance: float = 0
        self.account_number = account_number
        self.owner = owner
        self.balance = initial_balance
        self.transactions = []

    def deposit(self, amount: float) -> bool:
        if amount <= 0:
            print("Deposit rejected: amount must be strictly positive.")
            return False
        self.balance += amount
        self.transactions.append(f"DEPOSIT: +${amount:.2f}")
        return True

    def withdraw(self, amount: float) -> bool:
        if amount <= 0:
            print("Withdrawal rejected: amount must be strictly positive.")
            return False
        if amount > self.balance:
```

```
            print(f"Withdrawal rejected: Insufficient funds (Available: ${self.bal
            return False
        self.balance -= amount
        self.transactions.append(f"WITHDRAW: -${amount:.2f}")
        return True

    def get_summary(self) -> str:
        return f"Account [{self.account_number}] | Owner: {self.owner} | Balance:

acc = BankAccount("ACC-101", "Alice", 500.0)
acc.deposit(200.0)
acc.withdraw(150.0)
acc.withdraw(800.0)  # Should reject
print(acc.get_summary())
```

⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Method dispatch overhead is $\mathcal{O}(1)$ via CPython descriptor resolution.
- **Auxiliary Space:** Instance attribute memory allocation in `__dict__` vs `__slots__`.